

Dynamic Programming

Design and Analysis of Algorithms

Brian C. Dean

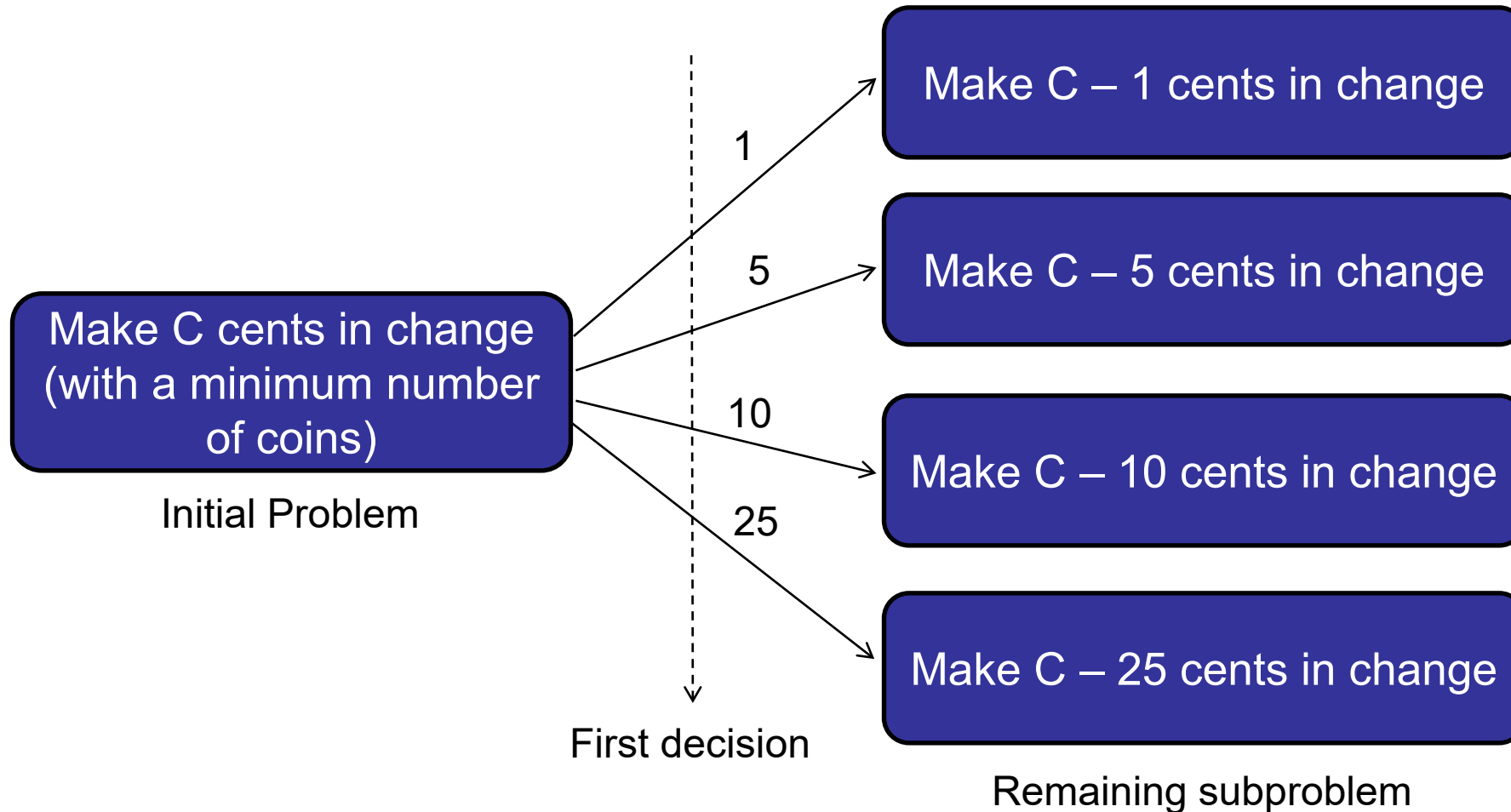
**School of Computing
Clemson University**



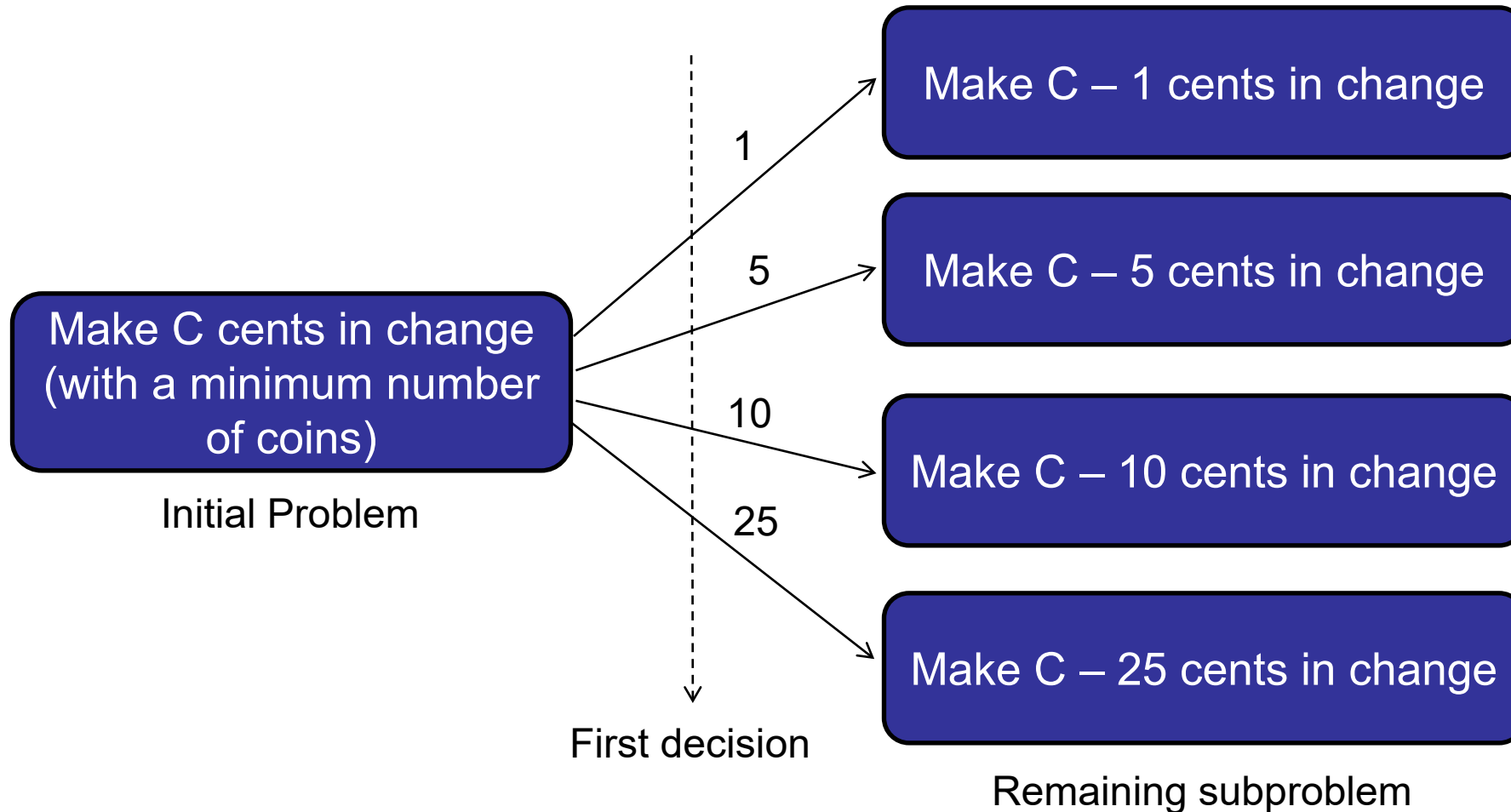
Making Change

- We have N different denominations of coins (e.g., 1 cent, 5 cent, 10 cent, 25 cent).
- We can use as many coins of each denomination as we wish.
- What is the minimum number of coins we need in order to construct exactly C cents worth of change?
- Greedy: use the largest denomination coin possible, repeat. (this works for the USA coin system, but not others)

Recall: Sequential Decisions

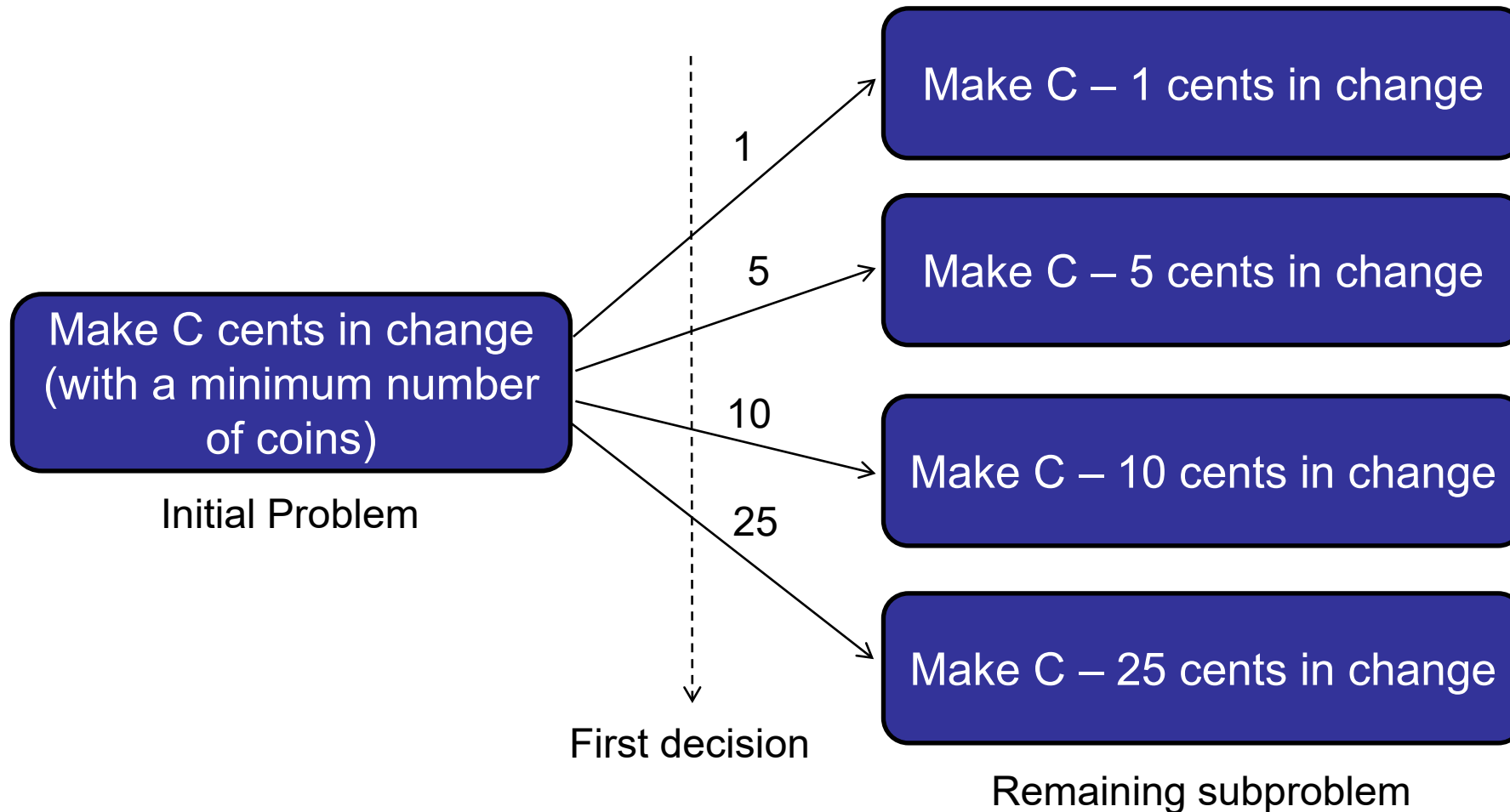


Optimal Substructure



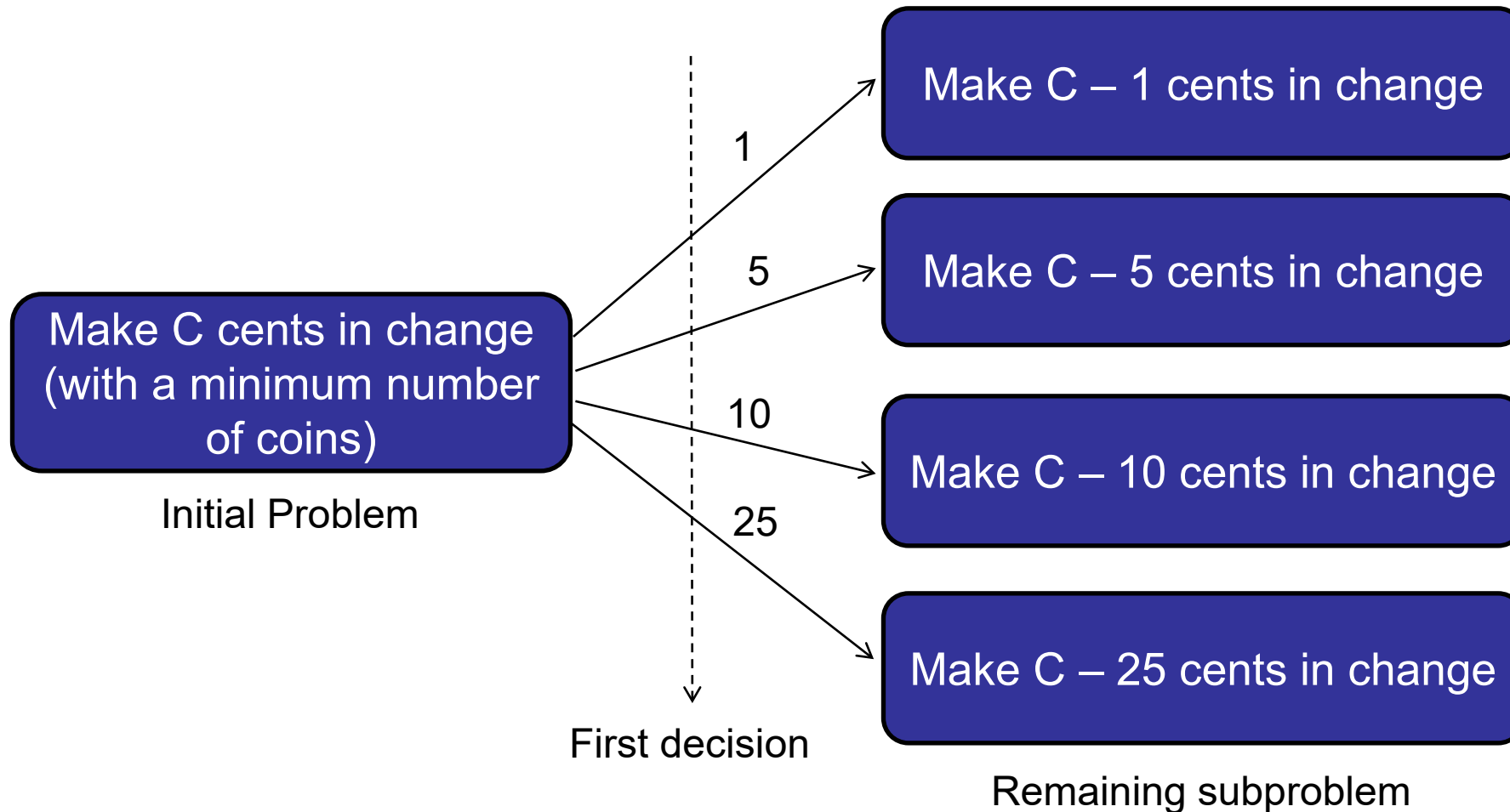
“Optimal substructure” property: residual problem remaining after 1st decision is of the same form as original – so optimally solving it will complete our solution.

Recall: Sequential Decisions



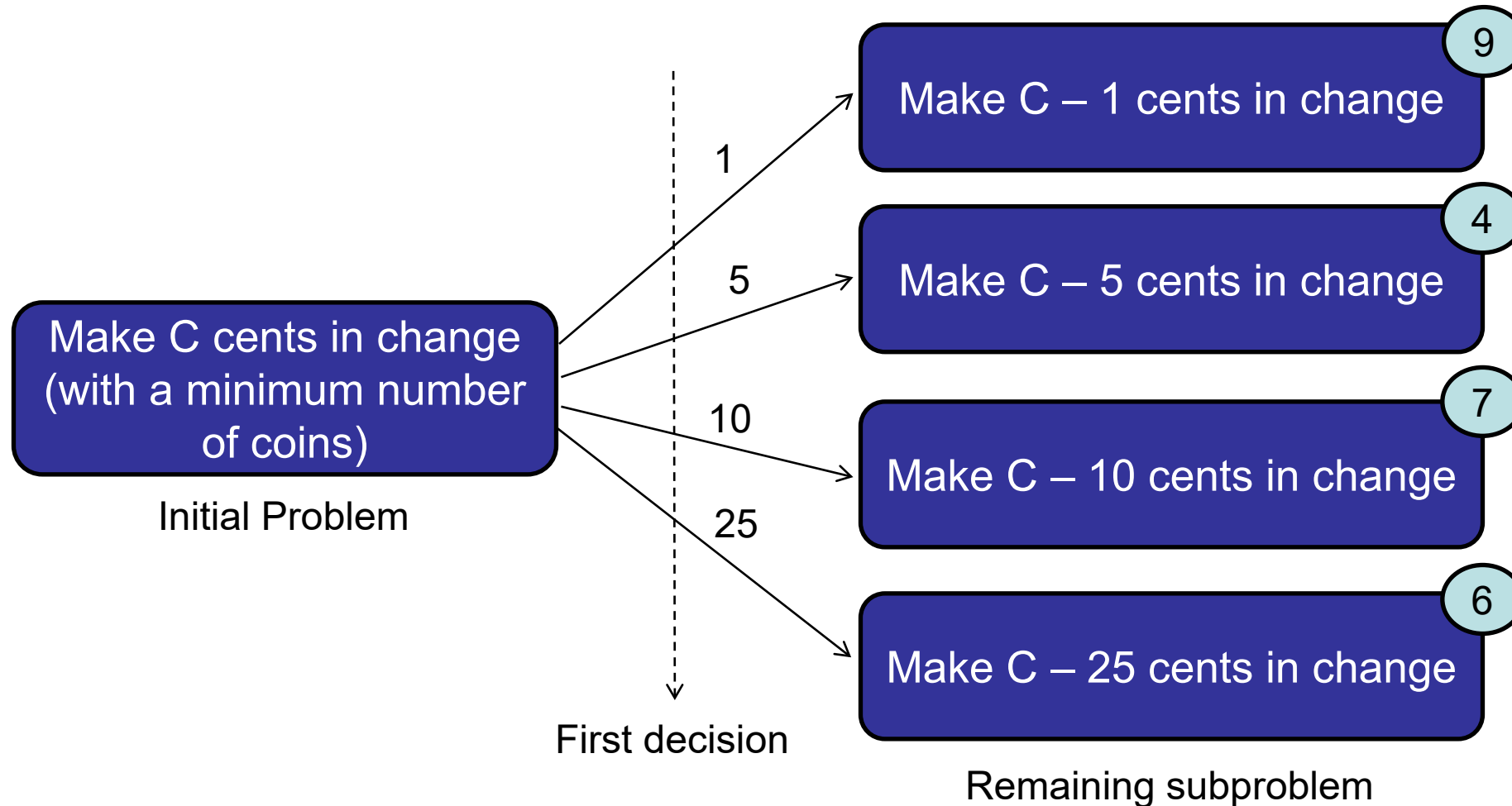
Greedy: Initial decision can be made safely and irrevocably made according to simple rule (e.g., use largest available coin).

Recall: Sequential Decisions



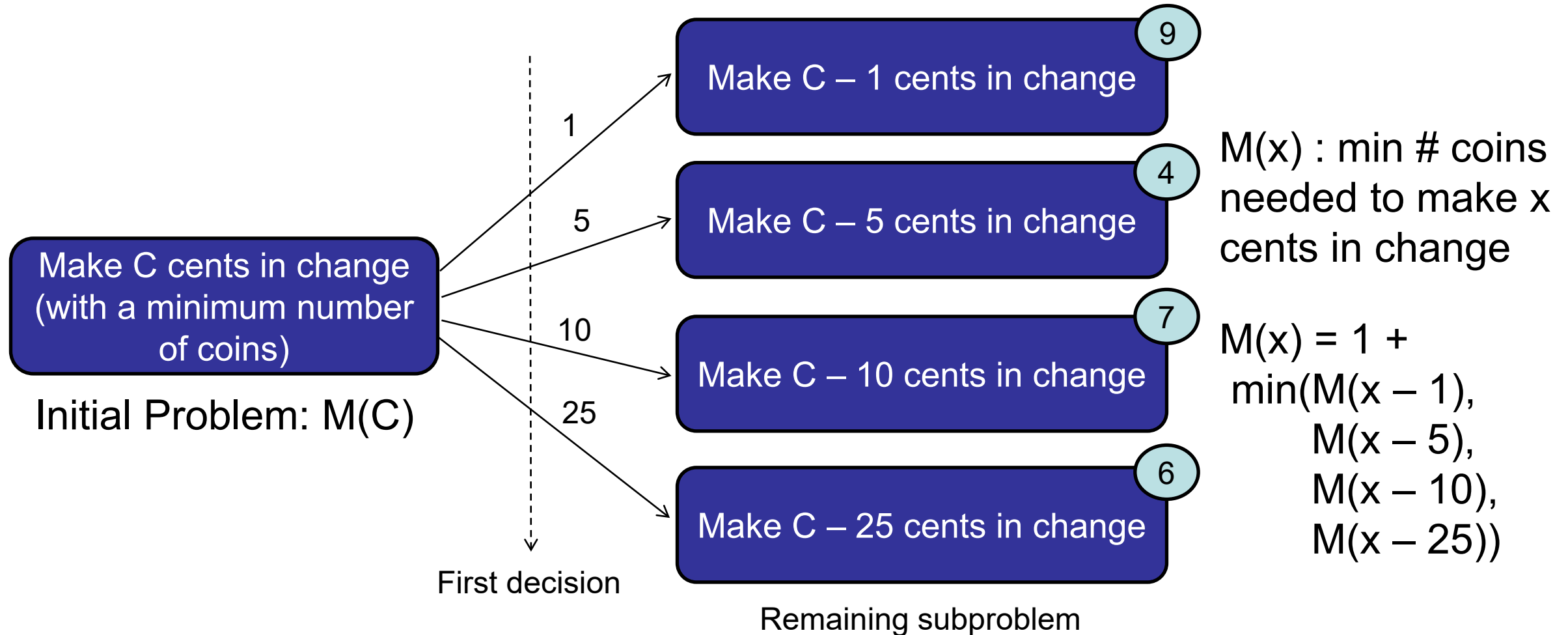
Exhaustive Search: Recursively try all possible first decisions, usually in some “greedy” order, usually with some sort of pruning heuristics to speed things up.

Sequential Decisions...



Dynamic Programming: Solve all possible subproblems from smallest to largest, making decisions easy!

Sequential Decisions...



Dynamic Programming: Solve all possible subproblems from smallest to largest, making decisions easy!

Computing Subproblem Solutions

- $M(x)$: min # of coins to make x cents in change
- $M(x) = 1 + \min(M(x - 1), M(x - 5), M(x - 10), M(x - 25))$
- Algorithm: apply above formula to compute $M(1) \dots M(C)$ in sequence, storing results in a table. Answer is $M(C)$.
- $O(C)$ total time. How much space needed?
- Don't forget base cases.

Bottom-Up Versus Top-Down

- Traditional $O(C)$ bottom-up approach:
For $x = 1 \dots C$:
$$M(x) = 1 + \min(M(x - 1), M(x - 5), M(x - 10), M(x - 25))$$
- Direct recursive top-down approach:
$$M(x) \{$$

if $x \leq 0$, return 0 as a base case
return $1 + \min(M(x - 1), M(x - 5), M(x - 10), M(x - 25))$
$$\}$$

Invoke by calling $M(C)$
- **Careful! As written, $\uparrow$ this has an exponential runtime!**

Fibonacci Numbers

- Traditional $O(N)$ bottom-up approach:

For $x = 1 \dots N$:

$$\text{Fib}(x) = \text{Fib}(x-1) + \text{Fib}(x-2)$$

- Direct recursive top-down approach:

$\text{Fib}(x) \{$

if $x \leq 1$, return 1 as a base case

return $\text{Fib}(x-1) + \text{Fib}(x-2)$

$\}$

Invoke by calling $\text{Fib}(N)$

- **Careful! As written, $\uparrow$ this has an exponential runtime!**

Important to Add “Memoization”

- Memoized recursive $O(N)$ top-down approach:

Create table $F[1 \dots N]$; set all entries to “uninitialized”

$\text{Fib}(x)$ {

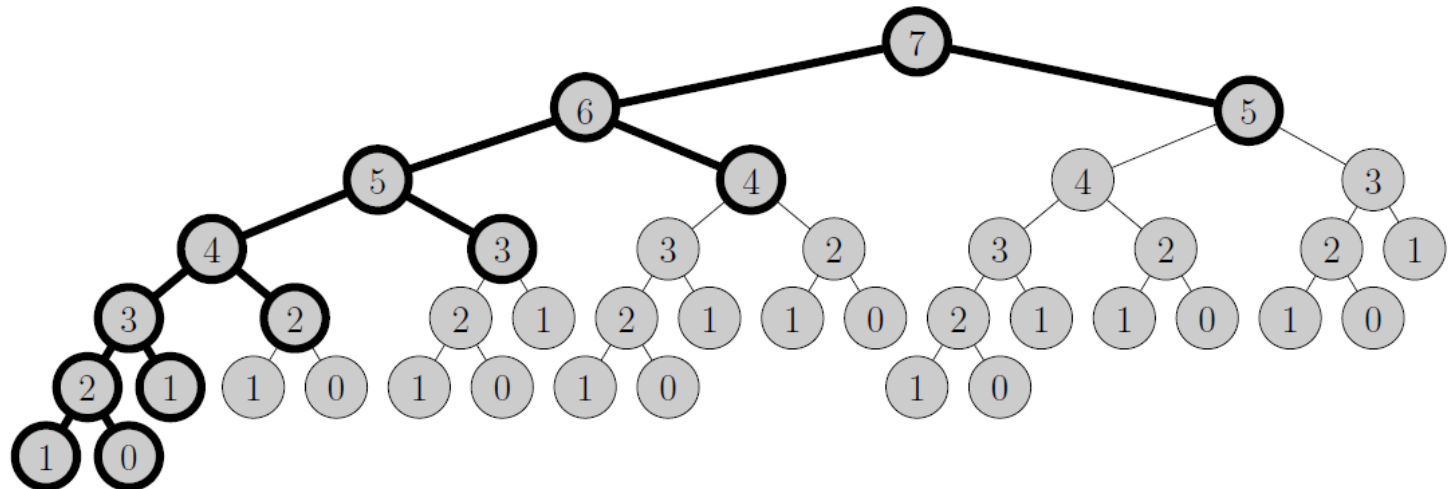
if $x \leq 1$, return 1 as a base case

If $F[n]$ uninitialized, set $F[n] = \text{Fib}(x-1) + \text{Fib}(x-2)$

return $F[n]$

}

Invoke by calling $\text{Fib}(N)$



Example: Longest Increasing Subsequence

- Given an array $A[1..n]$, what is the length of its longest increasing subsequence.
- E.g., 14 8 12 9 7 4 11 15 6 20 -3
- How do you find the structure of an optimal solution as well as its value? Follow a “traceback” path

Recall: Activity Selection

- Given intervals $[a_i, b_i]$, select a disjoint subset of as many intervals as possible.

Max-Length Activity Selection

- Given intervals $[a_i, b_i]$, select a disjoint subset of intervals of maximum total length.

When is DP Effective?

- Need optimal substructure in our problem.
- Typically only escalate to DP when greedy doesn't work.
- Number of distinct subproblems reasonably small.
 - E.g., with traveling salesman, DP won't help because there are exponentially many subproblems.
- Recursive approach would end up solving many subproblems multiple times, making memoization prudent.